МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФГБОУ ВО «Донской государственный технический университет»

Кафедра «Кибербезопасность информационных систем»

Лабораторная работа № 2

«Разработка Web-приложения для расчёта теоретической температуры горения»

РАЗРАБОТКА СТРАНИЦЫ РАСЧЁТА ТЕМПЕРАТУРЫ

Методические указания к выполнению
лабораторной работы по дисциплине
«Разработка защищённых Web-ресурсов»

Форма обучения: очная / очно-заочная

г. Ростов-на-Дону, 2026 г.

Цель работы: добавление к веб-приложению второй страницы, которая представляет собой форму ввода параметров дутья и вывод результата расчёта.

1. Предметная область

1.1. Теоретическая температура горения

В горне доменной печи кокс сгорает в потоке горячего дутья. **Теоретическая температура горения** (Тг) — расчётная температура продуктов сгорания в фурменном очаге при допущении, что всё выделившееся тепло идёт на нагрев этих продуктов, без учёта теплопотерь и теплообмена с шихтой.

Это ключевой технологический параметр. Низкая Тг ведёт к похолоданию горна и расстройству хода печи, чрезмерно высокая — к перегреву и ускоренному износу футеровки. Доменщик управляет ею через параметры дутья: температуру, влажность, содержание кислорода и расход вдуваемого природного газа. Именно эти зависимости отражены в справочной таблице на домашней странице приложения.

Точный расчёт требует полного теплового баланса. На практике для оперативного управления широко применяют эмпирические полиномиальные формулы.

1.2. Расчётная формула

```
Тг = 2405 + 0,75·(Тд − 1200) − 5,40·Wд + 43,00·(w − 21) − 52,00·Vпг
```

Обозначение	Параметр	Единица	Рабочий диапазон
Тд	Температура дутья	°C	900 … 1300
Wд	Влажность дутья	г/м³	1 … 40
w	Содержание кислорода в дутье	%	21 … 35

Обозначение	Параметр	Единица	Рабочий диапазон
Vпг	Объём вдуваемого природного газа	м³/100 м³ дутья	0 … 15

Источник: Расчёт теоретической температуры горения в горне доменной печи при вдувании пылеугольного топлива // КиберЛенинка. URL: cyber-leninka.ru/article/n/raschyot-teoreticheskoy-temperatury-goreniya-v-gorne-domennoy-pechi-pri-vduvanii-pyleugolnogo-topliva

1.3. Физический смысл коэффициентов

Прежде чем программировать формулу, убедитесь, что понимаете знаки:

+0,75 при Тд - более горячее дутьё вносит в очаг больше тепла;

−5,40 при Wд - влага разлагается с поглощением тепла (эндотермический процесс);

+43,00 при (w − 21) - обогащение кислородом сверх содержания в воздухе (21 %) уменьшает долю балластного азота, который приходится нагревать;

−52,00 при Vпг - разложение метана также эндотермично.

Понимание знаков пригодится при проверке результатов: если обогащение кислородом у вас понижает температуру, значит, в формуле ошибка.

7.4. Контрольные значения

№	Тд, °C	Wд, г/м³	w, %	Vпг, м³/100м³	Тг, °C	Режим
1	1000	10	21	0	**2201,0**	Базовый, без обогащения
2	1200	15	25	8	**2080,0**	Обогащение кислородом и ПГ
3	1100	20	23	5	**2048,0**	Промежуточный
4	1300	1	35	0	**3076,6**	Верхняя граница диапазонов

2. Расширение структуры проекта

Расчёт следует разместить не внутри обработчика запроса, а в отдельном модуле. Дополните структуру проекта до следующего вида:

```
FlaskDemo/
├── venv/                   ← виртуальное окружение (в PyCharm — .venv)
├── .gitignore
├── app.py                  ← контроллер (маршруты, обработка запросов)
├── requirements.txt
├── services/               ← пакет Python
│   ├── __init__.py
│   └── calculations.py     ← бизнес-логика (расчёт)
├── templates/              ← каталог шаблонов
│   ├── index.html
│   └── tempr.html
└── static/                 ← статические файлы
```

Создайте файл `.gitignore` со строками `venv/`, `.venv/` и `__pycache__/` — виртуальное окружение в репозиторий не помещают.

2.1. Зачем разделять слои

Расчёт выносится в отдельный пакет по трём причинам:

- **Тестируемость.** Функцию расчёта можно проверить без запуска Web-сервера.

- **Переиспользование.** Ту же функцию можно вызвать из консольной утилиты или планировщика.

- **Читаемость.** Контроллер отвечает за протокол HTTP, модуль расчёта - за предметную область. Смешение этих задач в одной функции быстро делает код неподдерживаемым.

3. Этап 1. Реализация бизнес-логики

Файл `services/calculations.py`:

```
def calculate_ttg(t_blast, humidity, oxygen, gas):
    """Теоретическая температура горения в горне доменной печи, °C.

    t_blast  -- температура дутья, °C
    humidity -- влажность дутья, г/м3
    oxygen   -- содержание кислорода в дутье, %
    gas      -- расход природного газа, м3/100 м3 дутья
    """
    return (2405
            + 0.75 * (t_blast - 1200)
            - 5.40 * humidity
            + 43.00 * (oxygen - 21)
            - 52.00 * gas)
```

Файл `services/__init__.py` оставьте пустым — он помечает каталог как пакет Python.

Проверьте расчёт до перехода к Web-части:

```
python -c "from services.calculations import calculate_ttg; \
    print(calculate_ttg(1000, 10, 21, 0))"
```

Ожидаемый вывод: `2201.0`. Проверьте и остальные три набора из п. 7.4. Это практическая иллюстрация тезиса о тестируемости, а именно слой бизнес-логики полностью проверен, когда Web-части ещё не существует.

4. Этап 2. Форма ввода параметров

Наполните содержимым шаблон `templates/tempr.html`, созданный вами в первой части работы. Соблюдайте те же правила семантической вёрстки, что и для домашней страницы.

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Теоретическая температура горения</title>
</head>
<body>
    <header>
        <h1>Теоретическая температура горения</h1>
        <nav><a href="/">На главную</a></nav>
    </header>

    <main>
        <form action="/tempr" method="POST">
            <label for="t_blast">Температура дутья, °C</label><br>
            <input type="text" id="t_blast" name="t_blast" required><br>

            <label for="humidity">Влажность дутья, г/м³</label><br>
            <input type="text" id="humidity" name="humidity" required><br>

            <label for="oxygen">Содержание кислорода, %</label><br>
            <input type="text" id="oxygen" name="oxygen" required><br>

            <label for="gas">Расход природного газа, м³/100 м³</label><br>
            <input type="text" id="gas" name="gas" required><br>

            <label for="comment">Комментарий к расчёту</label><br>
            <input type="text" id="comment" name="comment"><br><br>

            <input type="submit" value="Рассчитать">
        </form>

        {% if tempr %}
            <h2>Результат</h2>
            <p>Теоретическая температура горения:
               <b>{{ tempr }}</b> °C</p>
            <p>Комментарий: {{ comment }}</p>
        {% endif %}
    </main>

    <footer>
        <p>&copy; 2025 Имя Фамилия</p>
    </footer>
</body>
</html>
```

Разбор элементов формы:

- **`action`** - эндпоинт, куда отправляются данные;

- **`method="POST"`** - способ передачи; POST выбран потому, что данные передаются в теле запроса, а не в адресной строке;

- `name` - под этим именем значение поля попадёт в контроллер; ключом является именно `name`, а не `id`;

- `required` - проверка, выполняемая браузером до отправки формы;

`{% if tempr %}` - условный блок Jinja2. Результат выводится только тогда, когда он передан из контроллера. Именно поэтому при первом открытии страницы блок результата отсутствует.

Поле `comment` не участвует в расчёте, но выводится обратно на страницу. Такой сквозной путь данных «от пользователя - на страницу» встречается в реальных приложениях повсеместно.

5. Этап 3. Эндпоинт расчёта

Добавьте в `app.py` второй эндпоинт. Здесь и понадобится объект `request`, импортированный вами в первой части работы (рисунок 3).

```
from services import calculations


@app.route('/tempr', methods=['GET', 'POST'])
def tempr():
    if request.method == 'POST':
        t_blast = int(request.form['t_blast'])
        humidity = int(request.form['humidity'])
        oxygen = int(request.form['oxygen'])
        gas = int(request.form['gas'])
        comment = request.form['comment']

        result = calculations.calculate_ttg(t_blast, humidity,
                                            oxygen, gas)

        return render_template('tempr.html', tempr=result,
                               comment=comment)

    return render_template('tempr.html')
```

Один эндпоинт обслуживает оба метода, что задано списком `methods=['GET', 'POST']`. При GET (переход по ссылке из меню) возвращается пустая форма; при POST (отправка формы) - форма с результатом.

Данные формы извлекаются из объекта `request` по имени поля. Они приходят **строками** - протокол HTTP не передаёт информацию о типах, поэтому перед вычислением выполняется преобразование `int()`.

Передача результата в шаблон идёт по принципу «ключ - значение»: `tempr=result` в контроллере соответствует `{{ tempr }}` в шаблоне. Имена переменных совпадать не обязаны, но ключ и обращение к нему - должны.

Проверьте работу: перейдите на домашнюю страницу, воспользуйтесь ссылкой «Расчёт ТТГ» и выполните первый контрольный расчёт.

6. Этап 4. Журнал расчётов

Добавьте сохранение выполненных расчётов в файл. В `app.py`:

```
from datetime import datetime

LOG_PATH = 'static/calc_log.txt'


def log_calculation(form, result):
    with open(LOG_PATH, 'a', encoding='utf-8') as f:
        f.write(f"{datetime.now()} | "
                f"Тд={form['t_blast']} Wд={form['humidity']} "
                f"w={form['oxygen']} Vпг={form['gas']} | "
                f"Tг={result} | {form['comment']}\n")
```

Вызов разместите в ветке POST, после расчёта:

```
        log_calculation(request.form, result)
```

Журнал ведётся в человекочитаемом виде, то есть так, чтобы записи можно было просматривать без дополнительной обработки. Режим открытия `'a'` (append) означает дописывание в конец файла.

По завершению лабораторной работы №2 приложение должно выполнять расчёт теоретической температуры горения и сохранять его результат.

7. Контрольные вопросы

1. Чем GET-запрос отличается от POST-запроса? Почему для отправки параметров расчёта выбран POST?
2. Как данные из HTML-формы попадают в контроллер? Какой атрибут поля формы служит ключом?
3. Почему данные из формы приходят строками и что произойдёт, если пропустить преобразование типа?
4. Как значение передаётся из контроллера в шаблон? Что делает конструкция `{% if tempr %}` и почему она нужна?
5. Перечислите семантические теги, использованные в разметке, и обоснуйте знаки коэффициентов расчётной формулы с точки зрения физики процесса.

8. Перечень ссылок

1. Документация Flask: https://flask.palletsprojects.com/en/stable/
2. Методические указания о начале работы и установке Flask: https://rutube.ru/video/5d5132472bd7dca9e828d1b77cbba284
3. Jinja2 Template Designer Documentation: https://jinja.palletsprojects.com
4. Расчёт теоретической температуры горения в горне доменной печи при вдувании пылеугольного топлива // КиберЛенинка.
5. Гринберг М. Разработка веб-приложений с использованием Flask на языке Python. — М.: ДМК Пресс.